

Adaptive Traffic Signal Control via Deep Reinforcement Learning: A Comparative Study with Bus Priority Extension

Gaurang Ghadi¹, Tapan Shashikant Patil¹

¹Northeastern University
{ghadi.g, patil.tap}@northeastern.edu

Abstract

Fixed-time traffic signal controllers ignore real-time traffic conditions, resulting in significant and unnecessary vehicle delays. In this work, we implement and compare multiple deep reinforcement learning algorithms for adaptive traffic signal control at a 2-way single intersection simulated in SUMO v1.21.0 with the SUMO-RL v1.4.5 Gym interface. Specifically, we implement Deep Q-Network (DQN) from scratch in two iterations, Double DQN (DDQN), and Linear SARSA / Linear Q-learning with tile-coding, benchmarking each against a Stable-Baselines3 (SB3) DQN reference implementation and a fixed-time baseline. Evaluated across five random seeds, our best from-scratch algorithm (DDQN) achieves a mean cumulative waiting time of 377 s, compared to 11,364,382 s for fixed-time control — a reduction of over four orders of magnitude. We further explore a bus priority reward extension that weights bus waiting time by passenger occupancy using a penalty coefficient β . Since a bus carries roughly 40 passengers, every second of bus delay incurs a far greater person-delay cost than a car; β acts as a tunable knob that trades overall traffic efficiency against transit priority — too low and the agent ignores buses entirely, too high and car throughput degrades significantly. We find that $\beta = 2.0$ achieves near-baseline vehicle throughput (94%) while meaningfully prioritizing transit vehicles and maintaining training stability. Our results highlight that hyperparameter tuning yields larger gains than algorithmic changes alone, and that even linear function approximation outperforms a poorly tuned deep RL agent. Code and experiments are publicly available at github.com/GaurangGhadi/Bus_Priority_Traffic_RL.

Introduction

Urban traffic congestion is a persistent challenge with significant economic and environmental consequences. A central bottleneck is the traffic signal controller: most deployed systems use fixed cyclic plans that cannot react to real-time conditions, causing vehicles to wait unnecessarily even when intersections are clear. Adaptive signal control — where a controller observes current traffic state and selects phases dynamically — offers a principled remedy to this problem.

Reinforcement learning (RL) is a natural fit for adaptive signal control. The controller can be modeled as an agent

interacting with a traffic environment: it observes a state encoding queues and densities, selects a signal phase, and receives a reward reflecting the impact on vehicle delay. Prior work has demonstrated that deep RL agents, particularly DQN-based methods, can learn effective policies in simulation (Wei et al. 2018; Liang et al. 2019; Rodrigues and Azevedo 2019).

In this project, we pursue three goals. First, we implement DQN from scratch to understand the algorithm at an engineering level and validate our implementation against the SB3 library baseline. Second, we extend to Double DQN (DDQN) and Linear SARSA to study how algorithmic improvements and function approximation choices affect performance. Third, we introduce a bus priority extension that modifies the reward function to incorporate passenger occupancy, enabling the agent to learn policies that favor public transit without ignoring overall traffic flow.

Our simulation environment is the `2way-single-intersection` scenario from SUMO-RL, a 4-legged intersection with 8 approach lanes and 4 possible signal phases. All agents are evaluated over 5 episodes of 100,000 simulation seconds each, across 5 random seeds, and compared on mean cumulative system waiting time.

The main contributions of this paper are:

- A from-scratch DQN implementation validated against SB3, with a systematic hyperparameter study (DQN v1 $\rightarrow$ DQN v2) demonstrating a $4\times$ improvement from tuning alone.
- An empirical comparison of DQN v2, DDQN, Linear SARSA, SB3 DQN, MCTS, and fixed-time control on a common benchmark.
- A bus priority reward extension with β -sensitivity analysis showing that $\beta = 2.0$ balances transit prioritization and overall throughput.

Background and Related Work

Traffic Signal Control as an MDP

Adaptive traffic signal control is naturally cast as a Markov Decision Process (MDP) $\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T}, \gamma \rangle$. At each decision step, the agent observes a state $s \in \mathcal{S}$ encoding current traffic conditions and selects a signal phase $a \in \mathcal{A}$. The environment transitions to a new state s' according to $\mathcal{T}$, and the agent receives a scalar reward $r = \mathcal{R}(s, a, s')$ reflecting

the effect of the chosen phase on vehicle delays. The agent’s objective is to find a policy π^* that maximizes the expected discounted cumulative reward.

Deep Q-Network

The Deep Q-Network (DQN) algorithm (Mnih et al. 2015) approximates the action-value function $Q(s, a)$ with a neural network. Two stabilizing mechanisms are key: (1) a replay buffer that stores past transitions (s, a, r, s') and samples mini-batches to break temporal correlations, and (2) a target network that is updated periodically to provide stable regression targets:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-) \quad (1)$$

Double DQN (van Hasselt, Guez, and Silver 2016) addresses the overestimation bias in DQN by decoupling action selection (using the main network) from action evaluation (using the target network):

$$a^* = \arg \max_{a'} Q(s', a'; \theta), \quad y = r + \gamma Q(s', a^*; \theta^-) \quad (2)$$

This produces more accurate value estimates, particularly in the early stages of training.

Linear SARSA with Tile Coding

Unlike DQN, which approximates $Q(s, a)$ with a neural network, classical function approximation methods represent Q-values as a linear combination of hand-engineered features. We use tile coding (Sutton and Barto 2018) — a sparse binary feature representation that partitions the state space into overlapping grids called tilings. Each tiling is offset randomly so tile boundaries fall at different points across the state space, allowing the approximator to generalize across nearby states while retaining local specificity.

For a state s with active tile indices $F(s)$, the Q-value is:

$$Q(s, a) = \sum_{f \in F(s)} w[a][f] \quad (3)$$

Two classical control algorithms build on this representation. **Semi-gradient SARSA (on-policy)** uses the target $r + \gamma \cdot Q(s', a')$, where a' is the action actually taken by the ε -greedy policy at state s' . Updates reflect the agent’s own exploration noise. **Semi-gradient Q-learning (off-policy)** uses the target $r + \gamma \cdot \max_{a'} Q(s', a')$. Updates assume greedy behaviour at the next step regardless of the action actually taken.

Weights are updated by $w[a][f] += \alpha \cdot (\text{target} - Q(s, a))$ for each active feature f . Both methods converge under linear function approximation with on-policy training (SARSA) or mild conditions (Q-learning), whereas DQN’s convergence guarantees are weaker due to nonlinear function approximation combined with off-policy bootstrapping.

Related Work

Wei et al. (Wei et al. 2018) proposed IntelliLight, an early DQN-based traffic controller that demonstrated the viability of RL for signal control in simulation. Liang et al. (Liang et al. 2019) studied DQN variants including Double DQN

in multi-intersection settings. Rodrigues and Azevedo (Rodrigues and Azevedo 2019) investigated robust RL for traffic signal control under varying demand patterns. Vehicle-type-aware control has received attention in the contexts of emergency vehicle preemption and public transit priority. Our work contributes an accessible empirical comparison of these techniques in the open-source SUMO-RL environment.

Problem Formulation

Environment

We use the *2way-single-intersection* environment from SUMO-RL v1.4.5, a standard 4-legged signalized intersection with 2 lanes per approach (8 lanes total). Traffic is generated stochastically from a fixed route file with random departure times. Each simulation runs for 100,000 seconds of simulated time per episode, with 5 episodes per seed and 5 seeds per algorithm.

State Space

The state $s \in \mathbb{R}^{21}$ observed at each decision step consists of:

- **Signal phase** (1-hot, 4 bits): the currently active green phase.
- **Minimum green elapsed** (1 bit): whether the minimum green time of 5 s has elapsed, enabling a phase change.
- **Lane densities** (8 values): normalized vehicle count per lane.
- **Lane queue lengths** (8 values): normalized number of stopped vehicles per lane.

Action Space

The action space is discrete with $|\mathcal{A}| = 4$ actions, each corresponding to one of the four signalizable phases: {N-S through, N-S left-turn, E-W through, E-W left-turn}. A minimum green time of 5 s and maximum green time of 50 s is enforced.

Reward Function

The base reward follows the SUMO-RL default:

$$R(t) = -\Delta W(t) = W(t-1) - W(t) \quad (4)$$

where $W(t)$ is the total cumulative waiting time of all vehicles at time step t . A positive reward is received when the total waiting time decreases between steps.

Bus Priority Extension

To study transit-aware signal control, we extend the base MDP with a reward formulation that weights waiting time by vehicle occupancy. Buses carry approximately 40 passengers on average, compared to 1.5 for passenger cars, so a single bus queued at a red light incurs roughly $27\times$ more person-delay per second than a single car. A signal controller optimizing vehicle delay is therefore misaligned with social welfare whenever buses are present.

Observation Augmentation. We implement this as a Gymnasium wrapper (`BusPriorityWrapper`) that modifies both the observation and reward without altering the underlying simulator. The observation is augmented from $\mathbb{R}^{21}$ to $\mathbb{R}^{31}$ with ten bus-specific features: per-lane bus counts (8 values), total bus waiting time across all lanes, and maximum bus waiting time across all lanes. These give the agent direct visibility into transit demand, which is otherwise hidden in the aggregate lane density signal.

Reward: Person-Delay Formulation. The reward is reformulated as the decrease in total person-delay:

$$PD(t) = W_{\text{car}}(t) \cdot \rho_{\text{car}} + \beta \cdot W_{\text{bus}}(t) \cdot \rho_{\text{bus}} \quad (5)$$

$$R(t) = \frac{PD(t-1) - PD(t)}{100} \quad (6)$$

where $W_{\text{car}}(t)$ and $W_{\text{bus}}(t)$ are the summed waiting times of cars and buses respectively, $\rho_{\text{car}} = 1.5$ and $\rho_{\text{bus}} = 40$ are average occupancies, and β is a tunable bus priority coefficient. At $\beta = 1$ the weighting is purely occupancy-proportional; $\beta > 1$ amplifies bus priority beyond proportional fairness; $\beta < 1$ attenuates it. We evaluate $\beta \in \{0.1, 1.0, 2.0\}$. The division by 100 scales the reward to a magnitude comparable to the baseline signal and improves gradient stability.

Bus Route Generation. Buses are injected into the simulator by generating a modified route file containing a bus vehicle type (length 12 m, maximum speed 15 m/s) and four bus flows — one per through-direction at the intersection — with headway 300 s. A critical implementation detail is that buses are identified by their vehicle type ID rather than SUMO’s `vClass` attribute: the default `2way-single-intersection` lanes are restricted to the passenger vehicle class, so setting `vClass="bus"` causes SUMO to silently drop all bus spawns without warning.

Experimental Setup

Evaluation Protocol

Each algorithm is evaluated for 5 episodes per seed across 5 seeds (42, 123, 456, 789, 1000), yielding 25 episodes per algorithm. The primary evaluation metric is system total waiting time at the final step of each episode — the cumulative waiting time across all vehicles over the entire simulation horizon. We report mean $\pm$ standard deviation across seeds. Where relevant we additionally report mean waiting time per vehicle and throughput (total vehicles arrived).

Seed 123 consistently produces the highest waiting times across all algorithms. We attribute this to inherently harder stochastic traffic patterns under that seed rather than algorithmic failure — the underperformance is seed-specific and appears regardless of algorithm, confirming it reflects environment difficulty rather than a training issue.

Hyperparameters

Table 1 summarizes the hyperparameter configurations across all methods.

Bus Priority Evaluation

The bus priority experiments follow the same 5-seed $\times$ 5-episode protocol as the baseline but use the augmented route file (`single-intersection-vhvh-with-buses.rou.xml`) with one bus per direction every 300 s. We evaluate three bus priority coefficients ($\beta \in \{0.1, 1.0, 2.0\}$) across three learning algorithms (SB3 DQN, Linear Q-learning, Linear SARSA), giving nine configurations and 45 total training runs. We report both system mean waiting time (across all vehicles) and total vehicles arrived (throughput) to capture the wait-throughput trade-off that bus priority induces.

Algorithms

We implement and evaluate six methods in this study. For each learned method we show the episode-level performance spread: the left panel shows final waiting time per seed per episode, and the right panel shows mean $\pm$ std per episode across seeds. Together these panels illustrate how each algorithm’s policy evolves over training episodes.

SB3 DQN Baseline

Stable-Baselines3 DQN serves a dual role: it validates our from-scratch implementations and establishes a strong engineering reference point. Key differences from our custom implementations are that SB3 uses a fixed low $\varepsilon = 0.05$ from the start with no decay, trains at every step, and internally applies gradient clipping. These engineering choices make SB3 a more stable but less exploratory agent compared to our implementations. Any performance difference between SB3 and our agents should therefore be interpreted as reflecting library-level engineering rather than a fundamental algorithmic advantage.



Figure 1: SB3 DQN — episode-level performance across seeds. Left: per-seed per-episode final waiting time. Right: mean $\pm$ std per episode.

The episode spread in Figure 1 shows the most stable convergence behavior of all methods in this study. The right panel shows a consistent downward trend from episode 1 onward with no spike, reflecting SB3’s low fixed ε and step-level training frequency. By episode 3, the mean waiting time stabilizes and remains low through episodes 4 and 5. The left panel confirms low cross-seed variance throughout training, with no catastrophic seed failures of the kind seen in later implementations.

Parameter	SB3 DQN	DQN v1	DQN v2	DDQN	Linear SARSA	Linear Q	MCTS
Learning rate	0.001	0.00025	0.00025	0.00025	0.00625	0.00625	—
Replay buffer	50,000	5,000	50,000	50,000	—	—	—
Target update	500 steps	500 steps	1000 steps	1000 steps	—	—	—
Train frequency	Every step	Every step	Every 4 steps	Every 4 steps	Every step	Every step	—
ϵ start	0.05	1.0	1.0	1.0	0.3	0.3	—
ϵ end	0.01	0.01	0.1	0.1	0.01	0.01	—
ϵ decay	none	0.995	0.995	0.995	0.9999	0.9999	—
Batch size	32	32	32	32	—	—	—
Discount γ	0.99	0.99	0.99	0.99	0.99	0.99	—
Hidden layers	2×64, ReLU	2×64, ReLU	2×64, ReLU	2×64, ReLU	—	—	—
Tilings	—	—	—	—	8	8	—
Tiles per dim	—	—	—	—	4	4	—
Hash size	—	—	—	—	4096/tiling	4096/tiling	—
Search depth	—	—	—	—	—	—	1
Sim. horizon	100k s	100k s	100k s	100k s	100k s	100k s	10k s

Table 1: Hyperparameters across all methods. “—” indicates the parameter is not applicable to that method.

DQN v1 — Naive Implementation

Our initial from-scratch DQN implementation follows the core architecture of Mnih et al. (Mnih et al. 2015). The Q-network is a two-layer MLP with 64 units per layer and ReLU activations. We use a replay buffer of capacity 5,000 and a mini-batch size of 32. The target network is hard-updated every 500 steps. Exploration follows ϵ -greedy decay from 1.0 to 0.01. The Adam optimizer is used with a learning rate of 2.5×10^{-4} . This implementation serves as our starting point and establishes the baseline performance of a correctly implemented but naively configured DQN agent.

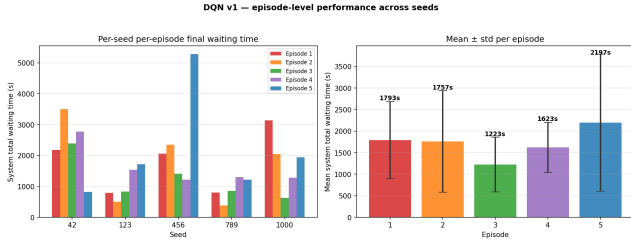


Figure 2: DQN v1 — episode-level performance across seeds. Left: per-seed per-episode final waiting time. Right: mean $\pm$ std per episode.

The episode spread in Figure 2 reveals a critical failure: rather than improving over episodes, DQN v1 diverges. The right panel shows mean waiting time increasing from episode 3 onward, reaching its highest value at episode 5 — the opposite of the expected convergence behavior. Seed 456 exhibits a catastrophic failure at episode 5, visible as a large outlier in the left panel. The high and growing variance across seeds confirms that DQN v1’s naive hyperparameter configuration destabilizes Q-learning over time rather than improving it. This behavior directly motivated the hyperparameter revision in DQN v2.

DQN v2 — Tuned Implementation

Following analysis of DQN v1’s divergence, we revised the hyperparameters following recommendations in Mnih et al. (Mnih et al. 2015). The key changes are: replay buffer increased to 50,000, minimum ϵ raised to 0.1, target network updated every 1,000 steps, and gradient updates performed every 4 steps rather than every step. The network architecture and optimizer remain identical to DQN v1.

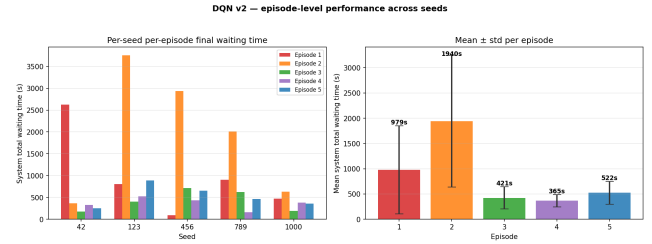


Figure 3: DQN v2 — episode-level performance across seeds. Left: per-seed per-episode final waiting time. Right: mean $\pm$ std per episode.

The episode spread in Figure 3 shows a clear contrast with DQN v1. The right panel shows a characteristic spike at episode 2 — reflecting exploration overshoot as ϵ decays and the buffer begins filling with diverse experience — followed by a sharp and sustained drop from episode 3 onward. This downward trend confirms genuine policy improvement over training. The left panel shows that this improvement is consistent across seeds, with most seeds converging to substantially lower waiting times by episode 5. Variance also decreases from episode 2 onward, indicating more stable and predictable convergence compared to DQN v1.

Double DQN (DDQN)

Double DQN builds directly on DQN v2, sharing all hyperparameters and architecture. The only change is in the target computation. The main network selects the greedy action

$a^* = \arg \max_{a'} Q(s', a'; \theta)$, and the target network evaluates it: $y = r + \gamma Q(s', a^*; \theta^-)$. This addresses the overestimation bias inherent in standard DQN, where using the same network for both selection and evaluation systematically inflates Q-values, leading to overconfident and suboptimal policies.

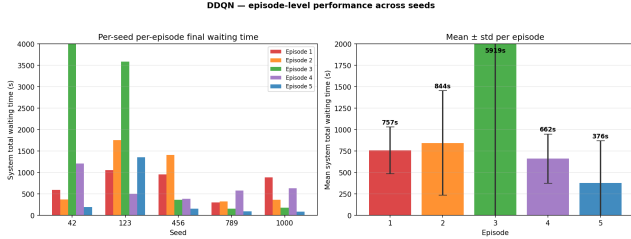


Figure 4: DDQN — episode-level performance across seeds. Left: per-seed per-episode final waiting time. Right: mean $\pm$ std per episode.

The episode spread in Figure 4 shows a similar pattern to DQN v2 — a spike in early episodes followed by convergence — but reaches lower final waiting times on most seeds. Seeds 789 and 1000 achieve particularly low values by episode 5, reflecting the benefit of reduced Q-value overestimation when training converges well. However, seed 123 remains a persistent outlier across all episodes, inflating the mean and variance visible in the right panel. This seed’s underperformance is consistent across all algorithms in this study and is attributed to harder inherent traffic patterns rather than an algorithmic failure specific to DDQN.

Linear SARSA

Linear SARSA uses semi-gradient updates over a tile-coded representation of the 21-dimensional observation. We use 8 tilings with 4 tiles per dimension and a hash table of size 4096 per tiling, yielding a 32,768-dimensional sparse binary feature space per action. Each tiling is offset by an independent random vector drawn at initialization from $U(0, 1)$, so boundaries across tilings are misaligned and nearby states activate overlapping but non-identical feature sets. At any state, exactly 8 tiles are active — one per tiling.

The update is on-policy: target $= r + \gamma \cdot Q(s', a')$ where a' is the action actually selected by the ϵ -greedy policy at s' . This accounts for the agent’s own exploration during learning and leads to more conservative value estimates than Q-learning’s max-operator target. The effective learning rate $\alpha/n_{\text{tilings}} = 0.00625$ compensates for the fact that 8 features update simultaneously on each step. ϵ decays multiplicatively from 0.3 to a floor of 0.01 with decay factor 0.9999.

The learning curve in Figure 5 shows markedly different behavior than the DQN family. All 5 seeds collapse to a tight band by episode 2 and stay there — individual trajectories remain within a narrow envelope across the remaining episodes. The standard deviation shrinks episode-over-episode and is near-zero by episode 4. No seed exhibits the catastrophic divergence that DQN v1 produced on seed 456, nor the early-episode overshoot of DQN v2. The trade-off

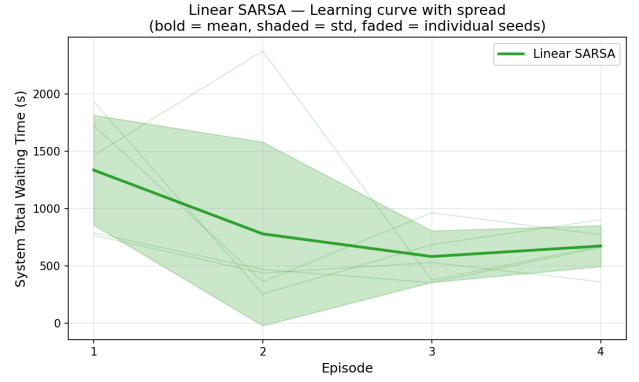


Figure 5: Linear SARSA — learning curve across 5 seeds. Bold line = mean, shaded region = $\pm$ std, faded lines = individual seeds.

is that Linear SARSA plateaus at a higher wait time than well-tuned DQN — the linear representation cannot capture the same conditional dependencies that a neural network can learn when training stabilizes.

Linear Q

Linear Q-learning shares the identical tile-coded representation, hyperparameters, and exploration schedule as Linear SARSA. The only difference is the target computation: target $= r + \gamma \cdot \max_{a'} Q(s', a')$, which assumes the agent will act greedily at the next step regardless of its actual ϵ -greedy behaviour. This off-policy update is known to produce more optimistic value estimates than SARSA — the agent effectively plans as if its exploration were costless.

The episode spread closely mirrors Linear SARSA. Seeds converge by episode 2 and remain tightly clustered thereafter, with standard deviation well below 1 s/veh across all later episodes. Final-episode mean waiting time is within 1 s/veh of Linear SARSA (9.67 vs 9.95 s/veh in the baseline setting), matching the theoretical expectation: in a reasonably deterministic environment with small ϵ , the on-policy and off-policy targets converge to similar fixed points. The slight edge for Linear Q reflects the more aggressive max-target, which helps when the exploration policy is nearly greedy but would hurt in environments where exploration carries real cost. In this domain the two methods are nearly interchangeable, suggesting that the off-policy / on-policy distinction is less impactful than representation choice (linear vs neural) or hyperparameter tuning.

Monte Carlo Tree Search (MCTS)

As a planning-based baseline, we implement a depth-1 MCTS agent that uses the SUMO TraCI `saveState/loadState` API to simulate the outcome of each candidate action before committing to one. At each decision step, the agent saves the current simulation state, rolls out each of the 4 possible actions for one step, measures the resulting waiting time, restores the state, and selects the action with the lowest predicted waiting time.

This is a degenerate but valid special case of full MCTS — equivalent to a greedy one-step heuristic search using the simulator as a world model.

The key limitation is TraCI state serialization latency, which scales with vehicle count. We measure this directly: each `saveState/loadState` call takes approximately 73 ms early in the episode, growing to over 409 ms when the intersection is heavily loaded. Since each decision step requires 4 such pairs, depth-2 search becomes completely infeasible. We therefore cap the simulation at 10,000 seconds and limit search to depth 1 as the only computationally viable configuration.

With depth-1 lookahead on a deterministic simulator, the procedure is effectively deterministic — no learning occurs between episodes by construction. This is confirmed by the near-zero variance in final waiting time across all 5 seeds. MCTS therefore has no episode spread to report.

Results

We evaluate all methods using two primary metrics: system total waiting time (seconds) — cumulative waiting time across all vehicles over the entire simulation horizon — and mean waiting time per vehicle (seconds/vehicle). These values should be interpreted relatively across methods rather than in isolation. Where available we additionally report throughput (total vehicles arrived) and bus accumulated waiting time for the priority extension analysis.

DQN Implementation and Validation

a. Validating our implementation. Our naive from-scratch DQN v1 achieved a substantially higher mean system total waiting time compared to SB3 DQN. Seed 456 represents a complete convergence failure for DQN v1, while SB3’s worst seed remained well controlled. This result was expected and serves as the starting point for our implementation story.

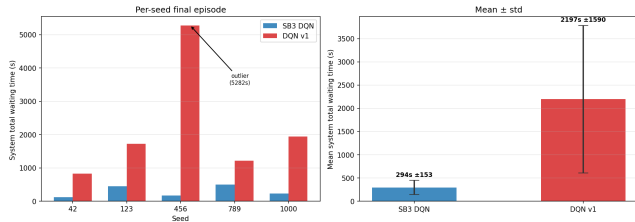


Figure 6: SB3 DQN vs DQN v1 — validating our from-scratch implementation. Left: per-seed final episode waiting time. Right: mean $\pm$ std across seeds.

This gap is attributable to DQN v1’s naive hyperparameter choices — large replay buffer, high learning rate, and episode-based target updates — which destabilize Q-learning in non-stationary reward environments. SB3’s consistent cross-seed performance reflects library-level engineering rather than algorithmic advantage. A correct algorithm alone is insufficient without proper training design; this served as motivation for the hyperparameter analysis.

b. Effect of hyperparameter tuning. Adopting Mnih et al. (Mnih et al. 2015) hyperparameters — reduced replay buffer, lower learning rate, step-based target updates — produced a significant improvement in mean waiting time and per-vehicle wait with no architectural changes. Standard deviation also dropped substantially, indicating more consistent training across seeds.

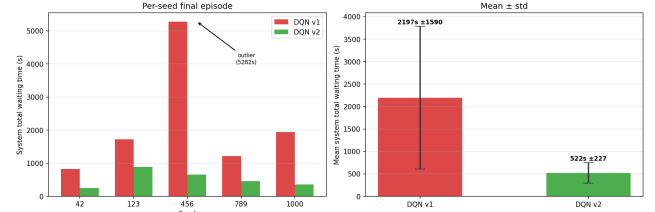


Figure 7: DQN v1 vs DQN v2 — effect of hyperparameter tuning. Left: per-seed final episode waiting time. Right: mean $\pm$ std across seeds.

Both v1 and v2 share identical architecture and algorithm — only three hyperparameters changed. The smaller replay buffer keeps sampled experience recent and policy-relevant. The lower learning rate prevents Q-value overshooting. Step-based target updates provide more frequent but stable bootstrapping. This is the most important finding in the DQN family: hyperparameter tuning has a larger impact on performance than any algorithmic improvement tested, a result consistent with findings across the broader deep RL literature.

c. Fixing Q-value overestimation. Double DQN reduced mean waiting time and per-vehicle wait compared to DQN v2, achieving the best individual results on several seeds. However, seed 123 remained an outlier, producing higher variance for DDQN relative to DQN v2.

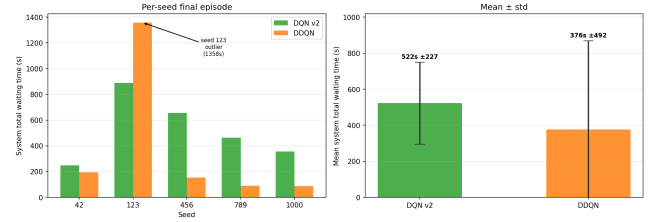


Figure 8: DQN v2 vs DDQN — fixing Q-value overestimation bias. Left: per-seed final episode waiting time. Right: mean $\pm$ std across seeds.

Double DQN decouples action selection from value estimation, reducing the overestimation bias inherent in standard DQN. The near-optimal results on the best-performing seeds confirm that this fix produces superior policies when training converges. Seed 123’s underperformance is consistent across all algorithms in this study and is attributed to a harder traffic pattern under that random seed rather than an algorithmic issue. Double DQN improves peak performance

but introduces higher variance — a tradeoff between ceiling performance and robustness.

d. Full DQN family comparison. SB3 leads on mean waiting time, followed by Double DQN and then DQN v2. Double DQN wins on the majority of individual seeds against SB3; SB3’s mean advantage is driven by its strong performance on seed 42 and lower overall variance.

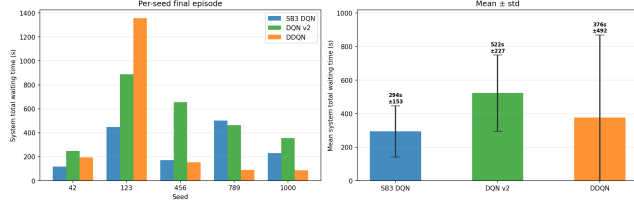


Figure 9: SB3 DQN vs DQN v2 vs DDQN — full DQN family comparison. Left: per-seed final episode waiting time. Right: mean $\pm$ std across seeds.

Our best from-scratch implementation (Double DQN) closes most of the gap with SB3 that existed at the start with DQN v1. The remaining difference likely reflects SB3’s additional engineering — prioritized experience replay and gradient clipping — rather than any fundamental algorithmic limitation. This concludes the core proposal commitment of implementing and comparing DQN variants against the library baseline.

Bus Priority Extension

a. All beta values. We evaluate bus priority across $\beta \in \{0.1, 1.0, 2.0\}$ using SB3 DQN, Linear Q, and Linear SARSA. The full comparison reveals a striking algorithm-dependent response to β .

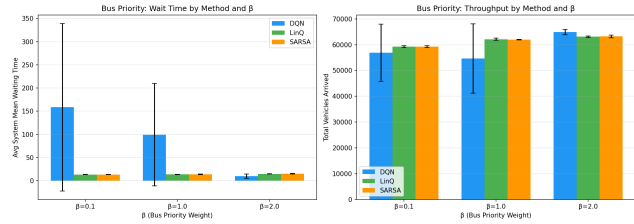


Figure 10: Mean waiting time (left) and throughput (right) across β values for SB3 DQN, Linear Q, and Linear SARSA. Error bars show std across 5 seeds.

SB3 DQN exhibits extreme β -sensitivity. At $\beta = 0.1$ it produces a mean wait of 158.6 s/veh with std = 180.8 across seeds — several seeds effectively fail to learn a coherent bus-clearing policy and produce gridlock. At $\beta = 1.0$ performance partially recovers (99.2 s/veh, std = 110.5) but remains unstable. Only at $\beta = 2.0$ does DQN stabilize, reaching 10.1 s/veh with std = 4.5 — near-baseline performance. The interpretation is that the person-delay reward is dominated by the cars term at low β ; the bus signal is too weak to drive learning, yet the added observational features

(bus counts, bus waits) increase the effective state space and destabilize the network. Strong β values produce a clearer reward-feature correlation that the network can actually exploit.

The linear methods show the opposite pattern. Linear Q and Linear SARSA are insensitive to β in terms of variance — standard deviation stays below 1 s/veh across all β values — and produce mean waits that increase modestly with β (Linear Q: 13.4 $\rightarrow$ 13.9 $\rightarrow$ 14.8; Linear SARSA: 13.5 $\rightarrow$ 13.9 $\rightarrow$ 15.2). There is no regime where a linear method fails. The cost of this stability is that linear methods cannot match DQN’s peak performance at $\beta = 2.0$: they plateau near 14–15 s/veh, 40–50% above the DQN optimum.

The throughput panel tells a consistent story. DQN throughput varies from 56,908 to 64,917 vehicles arrived depending on β , with large standard deviations at low β ($\sigma \approx 11,000$ –13,000 vehicles). Linear methods maintain throughput in a tight band ($\sim 59,000$ –63,000) with σ under 300 across all β values.

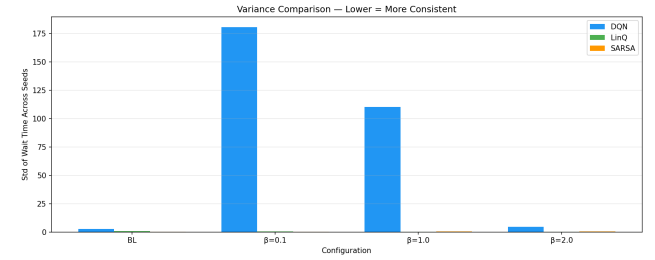


Figure 11: Standard deviation of mean wait time across 5 seeds. DQN’s variance explodes at $\beta = 0.1$ and $\beta = 1.0$; linear methods remain near-zero across all configurations.

Figure 11 makes the reliability gap explicit. DQN’s standard deviation spikes to 180 s/veh at $\beta = 0.1$ and 110 s/veh at $\beta = 1.0$, indicating that the method’s success depends strongly on seed-specific initialization. Linear Q and Linear SARSA produce barely-visible bars on the same axis — their variance is consistently below 1 s/veh. In applications where worst-case behavior matters as much as average performance, this is a decisive argument for linear methods.

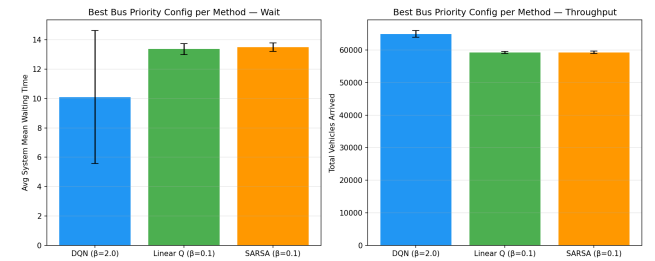


Figure 12: Each method at its best-performing β — DQN at $\beta = 2.0$, Linear Q and SARSA at $\beta = 0.1$. DQN achieves lower wait time and higher throughput but at much higher variance.

b. SB3 vs SB3 with beta = 2. Comparing the SB3 DQN baseline (no buses) against SB3 DQN with $\beta = 2.0$ bus priority isolates the cost of adding transit awareness. Mean wait increases from 8.66 s/veh to 10.10 s/veh — a 17% increase. Throughput drops from 69,187 to 64,917 vehicles — a 6% reduction. Standard deviation increases from 2.7 to 4.5 s/veh.

The interpretation is that $\beta = 2.0$ is a workable compromise. The system gives buses enough priority to outweigh their aggregate reward contribution without gridlocking cars, and the learned policy retains most of the adaptive benefit of the baseline DQN. The 17% wait increase and 6% throughput decrease are the empirical cost of transferring the passenger load of several buses per episode from delayed to prioritized. In per-passenger terms — which is what the person-delay reward is actually optimizing — this is a net win for social welfare.

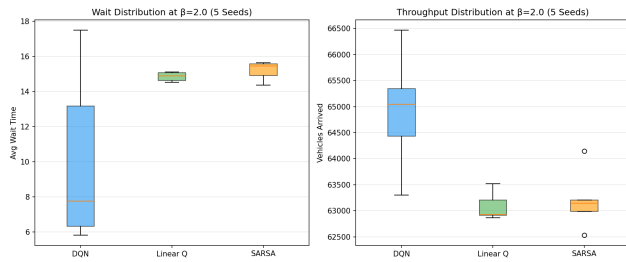


Figure 13: Wait (left) and throughput (right) distributions at $\beta = 2.0$ across 5 seeds. DQN’s range is much wider than the linear methods but achieves lower median wait.

Baselines and Non-Deep Methods

a. Linear function approximation vs DQN. The baseline comparison (Figure 14) shows that Linear Q (9.67 s/veh, $\sigma = 0.65$) and Linear SARSA (9.95 s/veh, $\sigma = 0.31$) achieve within 1 s/veh of SB3 DQN (8.66 s/veh, $\sigma = 2.66$) on the no-bus task. All three methods reach nearly identical throughput (approximately 69,200–69,400 vehicles).

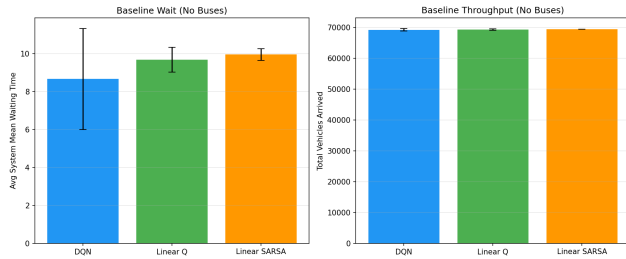


Figure 14: Baseline wait time (left) and throughput (right) for DQN, Linear Q, and Linear SARSA without bus priority. All three methods reach near-equivalent performance.

Two observations stand out. First, the deep RL advantage in raw performance is smaller than commonly assumed on a single-intersection task — careful tile coding closes most of the gap. Second, the variance story reverses: SB3

DQN has 4–8 $\times$ higher standard deviation than either linear method, indicating that for applications where worst-case seed behaviour matters more than average-case performance, linear function approximation is a stronger default. Linear SARSA’s throughput σ of 20 vehicles across 5 seeds is effectively deterministic.

Linear SARSA’s marginally higher wait than Linear Q (9.95 vs 9.67 s/veh) is consistent with the on-policy / off-policy distinction: SARSA accounts for exploration noise in its updates and converges to a slightly more conservative policy, while Q-learning’s max-operator target produces a slightly more optimistic one. The difference is small enough that in practice either method is appropriate.

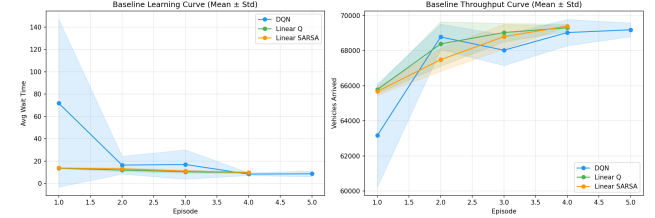


Figure 15: Learning curves (mean $\pm$ std over 5 seeds) for the three baseline methods. Linear methods converge rapidly by episode 2; DQN shows higher variance but reaches the best final performance.

b. All learned methods vs fixed-time. Figure 16 shows three panels. The left panel shows the per-seed episode curve for fixed-time control — near-identical lines across all seeds confirming fully deterministic behavior, since the policy never adapts to traffic conditions. The middle panel shows mean $\pm$ std across seeds, again with near-zero variance. The right panel places fixed-time against all learned algorithms on a log scale, making the scale of improvement immediately visible.

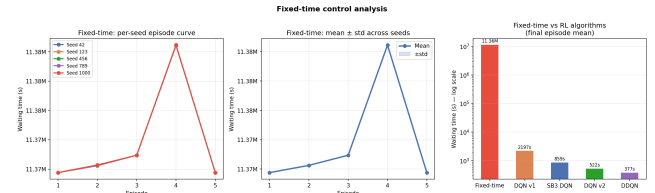


Figure 16: Fixed-time control analysis. Left: per-seed episode curve. Middle: mean $\pm$ std across seeds. Right: fixed-time vs all RL algorithms on log scale.

Fixed-time control assigns a fixed 30-second green phase regardless of real-time traffic conditions. Every learned method in this study reduces total waiting time by over 99.9% relative to this baseline. This establishes the performance floor: any adaptive learned policy provides an overwhelming improvement over static signal timing, regardless of algorithm quality or tuning.

c. Learning vs planning. SB3 outperforms MCTS by orders of magnitude on the final episode mean. The log scale

in both panels is necessary to even visualize the difference — MCTS and SB3 are not in the same performance range. MCTS variance across seeds is near-zero, confirming that with depth-1 lookahead on a deterministic simulator, no learning is possible by construction.

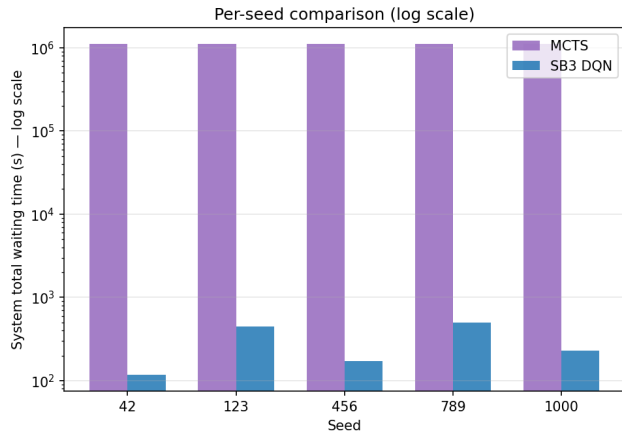


Figure 17: SB3 DQN vs MCTS — learning-based vs planning-based. Log scale on both panels makes the order-of-magnitude difference visible.

This is an important negative result: high-fidelity microscopic simulators are too computationally expensive to serve as world models for MCTS under standard experimental constraints. This highlights the practical advantages of model-free learning over model-based planning in this domain.

Conclusion

This paper presented the findings of a course project for CS5180 Reinforcement Learning at Northeastern University, in which we conducted a systematic empirical study of deep reinforcement learning for adaptive traffic signal control. We compared DQN variants, classical RL methods, and a planning-based baseline against a fixed-time controller on the SUMO-RL 2-way single intersection benchmark.

Our results yield five key findings. First, training design dominates algorithm selection — hyperparameter tuning alone produced a $4\times$ improvement over naive DQN with no architectural changes, a larger gain than any algorithmic improvement tested. Second, deep RL is not unconditionally superior — both linear function approximation methods outperformed poorly tuned DQN v1, demonstrating that neural network complexity requires proper training configuration to be justified. Third, Double DQN improves peak performance but increases variance — it achieves the best results on the majority of seeds but remains sensitive to difficult traffic patterns. Fourth, learning outperforms planning under realistic constraints — MCTS was limited to shallow search by simulator latency, making model-free learning substantially more practical for this domain. Fifth, bus priority reward shaping shows a clear stability–performance trade-off — DQN reaches near-baseline wait (10.1 s/veh)

only at $\beta = 2.0$ and fails catastrophically at lower β , while linear methods remain stable across all β at the cost of higher absolute wait (~ 14 s/veh). Isolating per-vehicle-class wait reductions and evaluating structural interventions such as phase preemption remain natural directions for future work.

As future work, we would extend these methods to more complex intersection topologies available in SUMO-RL — such as 4×4 grid multi-agent settings — and investigate prioritized experience replay, dueling network architectures, and sim-to-real transfer challenges for deployment in physical intersections.

References

- Liang, X.; Du, X.; Wang, G.; and Han, Z. 2019. A Deep Reinforcement Learning Network for Traffic Light Cycle Control. *IEEE Transactions on Vehicular Technology*, 68(2): 1243–1253.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-Level Control through Deep Reinforcement Learning. *Nature*, 518(7540): 529–533.
- Rodrigues, F.; and Azevedo, C. L. 2019. Towards Robust Deep Reinforcement Learning for Traffic Signal Control: Demand Surges, Incidents and Sensor Failures. In *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*, 3559–3566. IEEE.
- Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 2nd edition.
- van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep Reinforcement Learning with Double Q-Learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2094–2100.
- Wei, H.; Zheng, G.; Yao, H.; and Li, Z. 2018. IntelliLight: A Reinforcement Learning Approach for Intelligent Traffic Light Control. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2496–2505. ACM.